

Lab 13: Double Machine Learning and Causal Forests

SOCIOL 723 — Social Statistics II — Thursday, November 19

Wenhao Jiang

Table of contents

1	Goals	1
2	Regularization bias, in a world we control	2
3	DML with the DoubleML package	4
3.1	The interactive model (AIPW with ML)	5
3.2	Does the answer depend on the split?	6
4	Heterogeneous treatment effects	6
4.1	Sorted group ATEs, honestly	8
5	Exercises	10
6	Session information	10

1 Goals

1. **Watch naive ML fail** on a problem where we know the answer.
2. Fix it with orthogonalization, and fix it again with cross-fitting.
3. Run DML with `DoubleML`, and check that the answer does not depend on the split.
4. Estimate heterogeneous effects with a causal forest, and test whether the heterogeneity is real.

```
library(dplyr)
library(ggplot2)
library(glmnet)
library(ranger)
library(DoubleML); library(mlr3); library(mlr3learners)
library(grf)
lgr::get_logger("mlr3")$set_threshold("warn")
set.seed(723)
```

2 Regularization bias, in a world we control

The partially linear model $Y = \theta D + g(W) + \epsilon$, $D = m(W) + v$, with $\theta = 0.5$ known.

```
## approximately sparse coefficients: a few large, a long decaying tail
make_data <- function(n = 500, p = 200, theta = 0.5) {
  W <- matrix(rnorm(n * p), n, p); colnames(W) <- paste0("w", seq_len(p))
  b <- 1 / (seq_len(p))^2
  m <- as.vector(W %*% b)          # E[D | W]
  g <- as.vector(W %*% b)          # E[Y | W] beyond theta*D
  D <- m + rnorm(n)
  Y <- theta * D + g + rnorm(n)
  list(Y = Y, D = D, W = W)
}
```

Note the structure: the *same* covariates drive both D and Y , so W is genuinely confounding. With $p = 200$ and $n = 500$ OLS is estimable, but it is so high-variance and poorly conditioned as to be useless — which is why we regularize.

```
estimators <- function(dat) {
  Y <- dat$Y; D <- dat$D; W <- dat$W; n <- length(Y)

  ## 1. NAIVE: lasso Y on (D, W), leaving D unpenalized, and read off D
  pf <- c(0, rep(1, ncol(W)))
  f_all <- cv.glmnet(cbind(D, W), Y, penalty.factor = pf)
  naive <- as.vector(coef(f_all, s = "lambda.min"))[2]

  ## 2. ORTHOGONAL, no cross-fitting: residualize both on W, same data
  ly <- cv.glmnet(W, Y); lm_ <- cv.glmnet(W, D)
  ry0 <- Y - as.vector(predict(ly, W, s = "lambda.min"))
  rd0 <- D - as.vector(predict(lm_, W, s = "lambda.min"))
  ortho <- unname(coef(lm(ry0 ~ rd0))[2])

  ## 3. ORTHOGONAL + CROSS-FITTING
  K <- 5; folds <- sample(rep(1:K, length.out = n))
  ry <- rd <- numeric(n)
  for (k in 1:K) {
    tr <- folds != k; te <- folds == k
    a <- cv.glmnet(W[tr, ], Y[tr]); b <- cv.glmnet(W[tr, ], D[tr])
    ry[te] <- Y[te] - as.vector(predict(a, W[te, ], s = "lambda.min"))
    rd[te] <- D[te] - as.vector(predict(b, W[te, ], s = "lambda.min"))
  }
  dml <- unname(coef(lm(ry ~ rd))[2])
}
```

```
c(naive = naive, orthogonal = ortho, dml = dml)
}
```

```
round(estimators(make_data()), 4)
#>      naive orthogonal      dml
#>    0.6154     0.5369     0.5415
```

```
reps <- t(replicate(30, estimators(make_data())))
round(rbind(mean = colMeans(reps), bias = colMeans(reps) - 0.5,
            sd = apply(reps, 2, sd),
            rmse = sqrt(colMeans((reps - 0.5)^2))), 4)
#>      naive orthogonal      dml
#> mean 0.6020      0.5014 0.5062
#> bias 0.1020      0.0014 0.0062
#> sd   0.0494      0.0520 0.0485
#> rmse 0.1130      0.0512 0.0481
```

```
as.data.frame(reps) |>
  tidyr::pivot_longer(everything()) |>
  mutate(name = factor(name, levels = c("naive", "orthogonal", "dml"),
                        labels = c("naive", "orthogonal", "orthogonal + cross-fit"))) |>
  ggplot(aes(value, fill = name)) +
  geom_density(alpha = .5, colour = NA) +
  geom_vline(xintercept = 0.5, linetype = 2) +
  scale_fill_manual(values = c("firebrick", "goldenrod", "navy")) +
  labs(x = expression(hat(theta)), y = NULL, fill = NULL) +
  theme_minimal(base_size = 10)
```

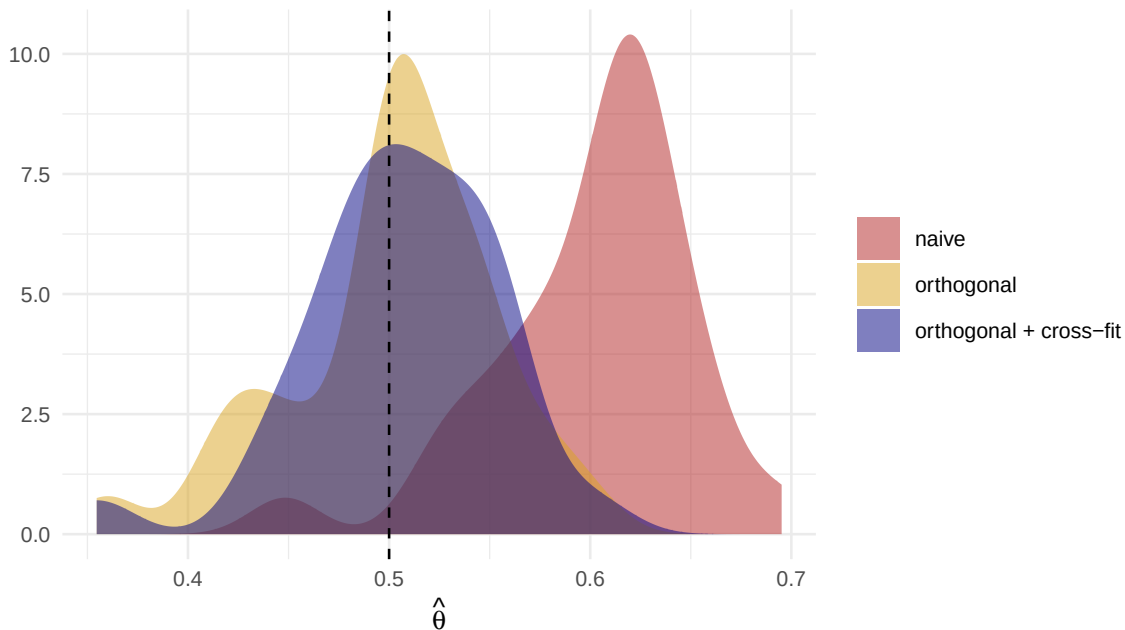


Figure 1: Sampling distributions over 30 simulations. The truth is 0.5 (dashed). The naive lasso is biased upward; residualizing both sides removes almost all of it.

! Important

Read the bias column. The naive estimator's error does not come from a bad learner — the lasso predicts Y well. It comes from the fact that *any* regularized estimate is shrunk, and that shrinkage transmits directly to $\hat{\theta}$ unless the moment is orthogonal.

Note also that here **orthogonalization does nearly all the work** and cross-fitting adds little: a cross-validated lasso does not overfit badly at this n . Cross-fitting matters much more with flexible learners such as forests and boosting, which fit the training sample far more aggressively — Exercise 2 makes that comparison.

3 DML with the DoubleML package

The 401(k) data: does *eligibility* for a 401(k) plan raise net financial assets? Eligibility is not random — it is offered by employers — so we condition on income, age, family size, education, and other plan participation.

```
## cached locally so the handout renders without a network connection
d401 <- readRDS("../Data/pension401k.rds")
dim(d401)
#> [1] 9915 12

dml_data <- DoubleMLData$new(d401, y_col = "net_tfa", d_cols = "e401",
                             x_cols = c("age", "inc", "educ", "fsize", "marr",
                                           "twoearn", "db", "pira", "hown"))
```

```

dml_data
#> ===== DoubleMLData Object =====
#>
#>
#> ----- Data summary -----
#> Outcome variable: net_tfa
#> Treatment variable(s): e401
#> Covariates: age, inc, educ, fsize, marr, twoearn, db, pira, hown
#> Instrument(s):
#> Selection variable:
#> No. Observations: 9915

lrn_g <- lrn("regr.ranger", num.trees = 400, min.node.size = 10)
lrn_m <- lrn("regr.ranger", num.trees = 400, min.node.size = 10)

set.seed(1)
plr <- DoubleMLPLR$new(dml_data, ml_l = lrn_g, ml_m = lrn_m,
                      n_folds = 5, n_rep = 1)

plr$fit()
plr$summary()
#> Estimates and significance testing of the effect of target variables
#>      Estimate. Std. Error t value      Pr(>|t|)
#> e401      9081      1293      7.02 0.00000000000022 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

## a naive OLS benchmark
ols <- lm(net_tfa ~ e401 + age + inc + educ + fsize + marr + twoearn + db +
          pira + hown, data = d401)
c(OLS = coef(ols)["e401"], DML_PLR = plr$coef)
#>      OLS.e401 DML_PLR.e401
#>      5896      9081

```

3.1 The interactive model (AIPW with ML)

The partially linear model assumes a constant effect. The *interactive* regression model does not — it is the AIPW score from Week 6 with cross-fitted ML nuisances.

```

lrn_g2 <- lrn("regr.ranger", num.trees = 400, min.node.size = 10)
lrn_m2 <- lrn("classif.ranger", num.trees = 400, min.node.size = 10,
             predict_type = "prob")

set.seed(1)
irm <- DoubleMLIRM$new(dml_data, ml_g = lrn_g2, ml_m = lrn_m2,

```

```

n_folds = 5, score = "ATE")
irm$fit()
irm$summary()
#> Estimates and significance testing of the effect of target variables
#>      Estimate. Std. Error t value      Pr(>|t|)
#> e401      7901      1167      6.77 0.000000000013 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

3.2 Does the answer depend on the split?

```

set.seed(2)
plr_rep <- DoubleMLPLR$new(dml_data, ml_l = lrn_g, ml_m = lrn_m,
                           n_folds = 5, n_rep = 10)
plr_rep$fit()
c(median_estimate = plr_rep$coef, se = plr_rep$se)
#> median_estimate.e401      se.e401
#>      9061      1311

```

`n_rep = 10` repeats the whole cross-fitting procedure with different partitions and reports the median. **If the estimate moves a lot across partitions, report that** — it means the nuisance estimates are unstable relative to the signal.

4 Heterogeneous treatment effects

```

X <- as.matrix(d401[, c("age", "inc", "educ", "fsize", "marr", "twoearn",
                        "db", "pira", "hown")])
Y <- d401$net_tfa
D <- d401$e401

cf <- causal_forest(X, Y, D, num.trees = 2000, seed = 1)
average_treatment_effect(cf, target.sample = "all")
#> estimate std.err
#>      7897      1141

```

```

test_calibration(cf)
#>
#> Best linear fit using forest predictions (on held-out data)
#> as well as the mean forest prediction as regressors, along
#> with one-sided heteroskedasticity-robust (HC3) SEs:
#>
#>      Estimate Std. Error t value      Pr(>t)

```

```
#> mean.forest.prediction      1.146      0.151      7.61 0.0000000000000015
#> differential.forest.prediction -0.158      0.410     -0.39      0.65
#>
#> mean.forest.prediction      ***
#> differential.forest.prediction
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

! Important

Read `test_calibration()` carefully. `mean.forest.prediction` near 1 says the average effect is well estimated. **`differential.forest.prediction` is the one that matters**: a coefficient near 1 with a small p -value is evidence of *real* heterogeneity; a coefficient near 0 says your CATE surface is noise, and you should not interpret it.

```
tau <- predict(cf)$predictions
tibble::tibble(tau, inc = d401$inc) |>
  filter(inc < quantile(inc, .98)) |>
  ggplot(aes(inc, tau)) +
  geom_point(alpha = .1, size = .4, colour = "navy") +
  geom_smooth(method = "loess", se = TRUE, colour = "firebrick") +
  geom_hline(yintercept = 0, linetype = 2) +
  labs(x = "income", y = expression(hat{tau}(x))) +
  theme_minimal(base_size = 10)
```

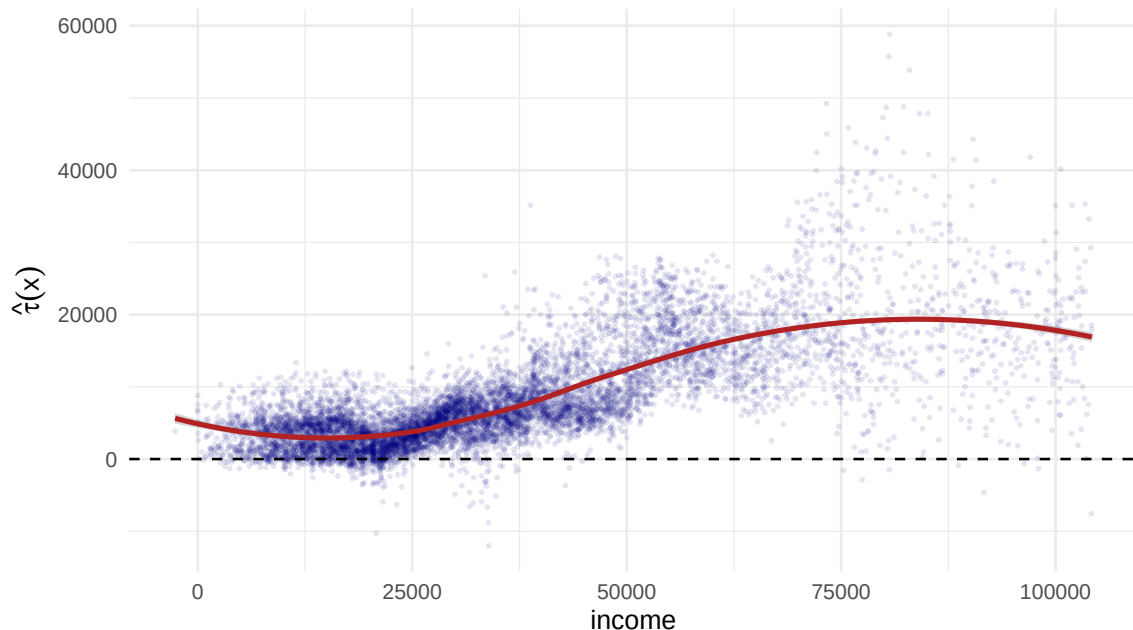


Figure 2: Estimated conditional average treatment effects against income.

4.1 Sorted group ATEs, honestly

Rank units by $\hat{\tau}$, form quintiles, and estimate a **doubly robust** ATE within each — using the forest’s own out-of-bag predictions so the ranking and the estimate do not use the same information.

```
dr <- get_scores(cf) # doubly robust scores, one per unit
grp <- cut(tau, breaks = quantile(tau, seq(0, 1, .2)), include.lowest = TRUE,
          labels = paste0("Q", 1:5))

gates <- tibble::tibble(dr, grp) |>
  group_by(grp) |>
  summarise(ate = mean(dr), se = sd(dr) / sqrt(n()), n = n(), .groups = "drop") |>
  mutate(lo = ate - 1.96 * se, hi = ate + 1.96 * se)
knitr::kable(gates, digits = 0)
```

grp	ate	se	n	lo	hi
Q1	5161	1944	1983	1351	8971
Q2	5274	857	1983	3595	6953
Q3	7250	2352	1983	2640	11861
Q4	9132	2212	1983	4797	13466
Q5	12667	4194	1983	4446	20887

```
ggplot(gates, aes(grp, ate)) +
  geom_pointrange(aes(ymin = lo, ymax = hi), colour = "navy") +
  geom_hline(yintercept = 0, linetype = 2) +
  labs(x = "quintile of predicted effect", y = "doubly robust ATE") +
  theme_minimal(base_size = 10)
```

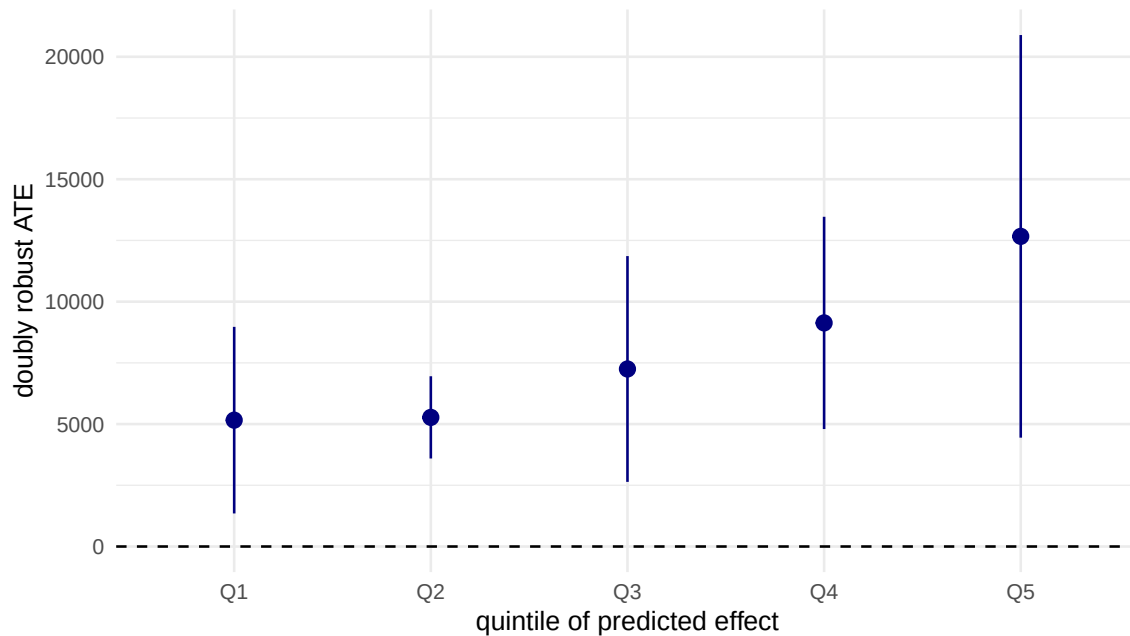



Figure 3: Doubly robust group average treatment effects by predicted-effect quintile.

```
## is the top quintile different from the bottom?
q1 <- dr[grp == "Q1"]; q5 <- dr[grp == "Q5"]
d_hat <- mean(q5) - mean(q1)
se_d <- sqrt(var(q5)/length(q5) + var(q1)/length(q1))
c(difference = d_hat, se = se_d, t = d_hat / se_d)
#> difference      se      t
#>  7505.894  4622.916  1.624
```

```
vi <- variable_importance(cf)
tibble::tibble(variable = colnames(X), importance = as.vector(vi)) |>
  arrange(desc(importance)) |> head(6) |> knitr::kable(digits = 3)
```

variable	importance
inc	0.635
age	0.169
educ	0.104
pira	0.027
fsize	0.027
twoearn	0.012

This is importance for **effect heterogeneity**, not for predicting the outcome — a different and more useful quantity. It is still descriptive: it says where the forest found variation, not why.

5 Exercises

1. **How slow can the learner be?** In `estimators()`, replace `cv.glmnet` with a deliberately crude learner — a single shallow tree via `ranger(x = as.data.frame(W), y = Y, num.trees = 1, max.depth = 2)`. Does DML still recover 0.5? At what point does the rate condition plausibly fail?
2. **Where cross-fitting earns its keep.** Repeat the three-estimator comparison with a random forest in place of the lasso (`ranger` on `as.data.frame(W)`, remembering that `ranger` needs column names). Forests fit their training sample far more aggressively than a cross-validated lasso, so the gap between the “orthogonal” and “orthogonal + cross-fit” columns should now be visible. Report both.
3. **Overlap.** In the 401(k) IRM, extract the estimated propensity scores and plot them by treatment status. Are there units with $\hat{e}$ near 0 or 1? Re-run trimming to $[0.05, 0.95]$ and report how the ATE moves.
4. **Learner choice.** Re-run the PLR with `regr.cv_glmnet` and with `regr.ranger`. How much does $\hat{\theta}$ move relative to its standard error? What does stability across learners buy you, and what does it not?
5. **DML for IV.** `fetch_401k(instrument = TRUE)` gives eligibility (`e401`) as an instrument for participation (`p401`). Fit DoubleMLPLIV and compare with the PLR estimate. Which estimand is each targeting?
6. **Honest heterogeneity.** The group ATEs above already use out-of-bag predictions and doubly robust scores, which removes most — but not all — of the optimism, because the same units informed the forest’s structure. Split the sample in half: fit the causal forest on the first half, predict $\hat{\tau}$ on the second, and estimate group ATEs in the second half only. How much does the top-minus-bottom contrast shrink?

6 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
```

```

#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#> [1] future_1.58.0      grf_2.5.0          mlr3learners_0.14.0
#> [4] mlr3_1.5.0         DoubleML_1.0.2     ranger_0.17.0
#> [7] glmnet_4.1-10      Matrix_1.7-0       ggplot2_4.0.0
#> [10] dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#> [1] gtable_0.3.6          shape_1.4.6.1      xfun_0.47
#> [4] lattice_0.22-7        vctrs_0.6.5        tools_4.4.1
#> [7] generics_0.1.3        sandwich_3.1-1     parallel_4.4.1
#> [10] tibble_3.2.1          fansi_1.0.6        pkgconfig_2.0.3
#> [13] data.table_1.18.2.1   checkmate_2.3.2    RColorBrewer_1.1-3
#> [16] S7_0.2.0              mlr3tuning_1.6.0   uuid_1.2-1
#> [19] lifecycle_1.0.4       compiler_4.4.1     farver_2.1.2
#> [22] tinytex_0.53          codetools_0.2-20   readstata13_0.11.0
#> [25] htmltools_0.5.8.1    bbotk_1.9.0        yaml_2.3.10
#> [28] tidyr_1.3.1           pillar_1.9.0       crayon_1.5.3
#> [31] MASS_7.3-60.2         clusterGeneration_1.3.8 iterators_1.0.14
#> [34] foreach_1.5.2         nlme_3.1-164       parallelly_1.45.0
#> [37] tidyselect_1.2.1      digest_0.6.37      mvtnorm_1.3-1
#> [40] purrr_1.0.2           listenv_0.9.1      mlr3measures_1.2.0
#> [43] labeling_0.4.3        splines_4.4.1      fastmap_1.2.0
#> [46] grid_4.4.1            cli_3.6.5          magrittr_2.0.3
#> [49] paradox_1.0.1         survival_3.6-4     utf8_1.2.4
#> [52] future.apply_1.20.0   withr_3.0.1        scales_1.4.0
#> [55] backports_1.5.0       rmarkdown_2.28     globals_0.18.0
#> [58] zoo_1.8-12            evaluate_1.0.0     knitr_1.48
#> [61] lmtest_0.9-40         mgcv_1.9-1         mlr3misc_0.21.0
#> [64] rlang_1.3.0           Rcpp_1.1.1         glue_1.7.0
#> [67] palmerpenguins_0.1.1 jsonlite_1.8.9     lgr_0.4.4
#> [70] R6_2.5.1

```